

Операционные системы: функции ОС и архитектуры ядра

Конспект по базовым функциям операционной системы, управлению ресурсами и сравнению монолитной, модульной и микроядерной архитектур.

Структура материала

1. ОС как управляющий слой
2. Процессы, память, файлы и устройства
3. Монолитная архитектура ядра
4. Модульное ядро как компромисс
5. Микроядро и критерии выбора архитектуры

ОС как управляющий слой

Операционная система — это комплекс программ, который организует работу вычислительной системы и управляет её ресурсами. Она скрывает детали оборудования и предоставляет пользователю и приложениям единый способ взаимодействия с процессором, памятью, файлами и устройствами.

На базовом уровне ОС решает две большие задачи. Во-первых, распределяет ограниченные ресурсы между программами. Во-вторых, делает работу с компьютером удобной и безопасной за счёт стандартных интерфейсов и правил доступа.

Такой взгляд помогает связать между собой все функции ОС: управление процессами, памятью, файлами и устройствами перестают быть отдельными темами и складываются в единую систему координации.

КЛЮЧЕВЫЕ ТЕЗИСЫ

- Управление процессами определяет, какие программы выполняются и когда получают процессорное время.
- Управление памятью отвечает за размещение данных и защиту адресных пространств.
- Файловая подсистема обеспечивает хранение, имена, каталоги и права доступа.
- Подсистема ввода-вывода связывает программы с периферийными устройствами.
- Пользовательский интерфейс даёт человеку точку входа в систему: оболочку, терминал или графическую среду.

Процессы, память, файлы и устройства

Управление процессами включает создание, завершение, переключение контекста и планирование. ОС должна сделать так, чтобы несколько программ могли поочерёдно использовать процессор, не мешая друг другу.

С памятью ОС работает на уровне выделения, защиты и освобождения областей. Изоляция адресных пространств защищает программы от случайного повреждения чужих данных и повышает устойчивость системы.

Файловая система и драйверы устройств дополняют эту картину. Пользователь видит файлы и каталоги, а внутри ОС идёт работа с блоками, буферами, очередями запросов и аппаратными контроллерами.

КЛЮЧЕВЫЕ ТЕЗИСЫ

- Планировщик выбирает следующий процесс с учётом политики и приоритетов.
- Виртуальная память позволяет программам работать с адресным пространством, большим, чем объём оперативной памяти.
- Файловая система должна поддерживать целостность данных, имена, метаданные и права доступа.
- Драйверы превращают универсальные системные вызовы в команды конкретным устройствам.

Монолитная архитектура ядра

Монолитное ядро объединяет основные подсистемы ОС в один большой исполняемый блок, который работает в привилегированном режиме. Драйверы, файловые системы и сетевые подсистемы имеют прямой доступ друг к другу и к ядру.

Главное достоинство монолита — производительность. Подсистемы взаимодействуют без лишних переключений между режимами и без сложного обмена сообщениями.

Главный риск тоже очевиден: ошибка в драйвере или внутреннем модуле может повредить всё ядро и привести к падению системы. Поэтому монолит обычно выбирают там, где критична скорость и хорошо контролируется набор компонентов.

КЛЮЧЕВЫЕ ТЕЗИСЫ

- Монолитное ядро быстро из-за прямых вызовов внутри общего адресного пространства.
- Сбой в одном компоненте может повлиять на всю систему.
- Такой подход подходит для систем, где производительность важнее максимальной изоляции.
- Классический пример — Linux как ядро монолитного типа с развитыми внутренними подсистемами.

Модульное ядро как компромисс

Модульная архитектура остаётся монолитной по сути, но позволяет загружать и выгружать части ядра динамически. Это касается драйверов, файловых систем и некоторых сетевых или сервисных расширений.

Такой подход сохраняет высокую производительность, потому что модули всё равно работают в привилегированном режиме и в общем адресном пространстве. При этом сопровождение становится удобнее: часть функциональности можно обновить или подключить без полной перекомпиляции ядра.

Но модульность не делает систему микроядерной. Ошибка внутри загруженного модуля по-прежнему способна повредить ядро, потому что строгой изоляции между ними нет.

КЛЮЧЕВЫЕ ТЕЗИСЫ

- Модульное ядро гибче чисто монолитного из-за динамической загрузки компонентов.
- Производительность близка к монолитной, потому что модули не выносятся в пользовательский режим.
- С точки зрения отказоустойчивости модульность лучше сопровождать строгой проверкой качества драйверов.
- Linux с модулями и многие современные серверные ОС хорошо иллюстрируют этот компромисс.

Микроядро и критерии выбора архитектуры

Микроядерная архитектура оставляет внутри ядра только минимальный набор функций: планирование, базовое управление памятью и механизм межпроцессного взаимодействия. Драйверы, файловые системы и сетевые службы выносятся в пользовательский режим и работают как отдельные процессы.

Главный плюс микроядра — надёжность и изоляция. Если драйвер падает, его проще перезапустить без остановки всей системы. Это особенно важно для встраиваемых и критичных систем, где устойчивость важнее абсолютной скорости.

Минус микроядерного подхода — дополнительный накладной расход на обмен сообщениями и переключения между режимами. Поэтому архитектуру выбирают не 'вообще лучшую', а подходящую под класс задач: производительность, безопасность, возможность обновления и требования к отказоустойчивости.

КЛЮЧЕВЫЕ ТЕЗИСЫ

- Микроядро минимизирует код в привилегированном режиме.
- Сервисы в пользовательском режиме легче изолировать, перезапускать и тестировать.
- Дополнительный IPC делает такую архитектуру медленнее чистого монолита.
- QNX и Minix обычно приводят как учебные и промышленные примеры микроядерного подхода.
- Для настольных и серверных систем часто выбирают монолит или модульный вариант, для критичных встраиваемых систем — микроядро.